

CITS5017 Deep Learning

Topics 6&7: Convolutional AEs, Denoising AEs, Variational AEs, GANs, and DCGANs

Associate Professor Du Huynh
(Unit Coordinator and Lecturer)

Semester 2, 2026



THE UNIVERSITY OF
WESTERN
AUSTRALIA

Chapters 17.

Hands-on Machine Learning with Scikit-Learn, Keras & TensorFlow



Autoencoders, GANs, and Diffusion Models

Here are the main topics we will briefly cover:

- Convolutional autoencoders
- Denoising autoencoders
- Variational autoencoders
- Generative Adversarial Networks
- Deep convolutional generative adversarial networks (DCGANs)

Convolutional Autoencoders

If you are dealing with images, then the autoencoders we have seen so far will not work well. So if you want to build an **autoencoder for images** (e.g., for *unsupervised pretraining* or *dimensionality reduction*), you will need to build a **convolutional autoencoder**.

The **encoder** of a convolutional autoencoder is just a regular CNN composed of convolutional layers and pooling layers. It typically reduces the spatial dimensionality of the inputs (i.e., height and width) while increasing the depth (i.e., number of channels).

The **decoder** must do the reverse operation (upscale the image and reduce its depth back to the original dimensions), and for this you can use **transpose convolutional layers** (alternatively, you could combine upsampling layers with convolutional layers).

Convolutional Autoencoders (cont.)

Here is a simple convolutional autoencoder for Fashion MNIST:

```
conv_encoder = tf.keras.Sequential([
    tf.keras.layers.Reshape([28, 28, 1]),
    tf.keras.layers.Conv2D(16, 3, padding="same", activation="relu"),
    tf.keras.layers.MaxPool2D(pool_size=2), # output: 14 x 14 x 16
    tf.keras.layers.Conv2D(32, 3, padding="same", activation="relu"),
    tf.keras.layers.MaxPool2D(pool_size=2), # output: 7 x 7 x 32
    tf.keras.layers.Conv2D(64, 3, padding="same", activation="relu"),
    tf.keras.layers.MaxPool2D(pool_size=2), # output: 3 x 3 x 64
    tf.keras.layers.Conv2D(30, 3, padding="same", activation="relu"), # output: 3 x 3 x 30
    tf.keras.layers.GlobalAvgPool2D() # output: 30
])
conv_decoder = tf.keras.Sequential([
    tf.keras.layers.Dense(3 * 3 * 16), # 144 neurons
    tf.keras.layers.Reshape((3, 3, 16)), # output: 3 x 3 x 16
    tf.keras.layers.Conv2DTranspose(32, 3, strides=2, activation="relu"), # output: 7 x 7 x 32
    tf.keras.layers.Conv2DTranspose(16, 3, strides=2, padding="same",
                                    activation="relu"), # output: 14 x 14 x 16
    tf.keras.layers.Conv2DTranspose(1, 3, strides=2, padding="same"), # output: 28 x 28 x 1
    tf.keras.layers.Reshape([28, 28])
])
conv_ae = tf.keras.Sequential([conv_encoder, conv_decoder])

# compile and fit the model
conv_ae.compile(loss="mse", optimizer="nadam")
history = conv_ae.fit(X_train, X_train, epochs=10,
                     validation_data=(X_valid, X_valid))
```

Convolutional Autoencoders (cont.)

Inspect the `conv_decoder` model:

```
>>> conv_decoder.summary()
Model: "sequential_12"

-----
Layer (type)                Output Shape                Param #
-----
dense_12 (Dense)            (None, 144)                 4464
reshape_4 (Reshape)         (None, 3, 3, 16)           0
conv2d_transpose (Conv2DTra (None, 7, 7, 32)           4640
nspose)
conv2d_transpose_1 (Conv2DT (None, 14, 14, 16)         4624
ranspose)
conv2d_transpose_2 (Conv2DT (None, 28, 28, 1)         145
ranspose)
reshape_5 (Reshape)         (None, 28, 28)             0
=====
Total params: 13,873
Trainable params: 13,873
Non-trainable params: 0

>>> conv_ae.count_params() # output the total number of parameters (trainable + non-trainable)
54479
```

Other Types of Autoencoders

So far we have seen various kinds of autoencoders:

- basic, stacked, and convolutional.

We have looked at how to train them :

- either in one shot or in several phases.

We have also looked at a couple of applications:

- **data visualization** and **unsupervised pretraining**

Up to now, in order to force the autoencoder to learn interesting features, we have limited the size of the coding layer, making it *undercomplete*.

There are actually many other kinds of constraints that can be used, including ones that allow the coding layer to be just as large as the inputs, or even larger, resulting in an *overcomplete autoencoder*.

Let's look at *denoising autoencoders* and *variational autoencoders* in the next few slides.

Denoising Autoencoders

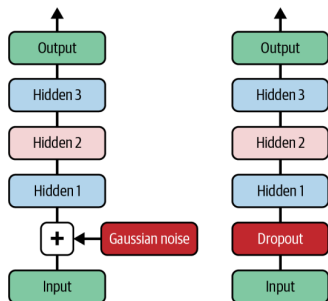


Figure 17-8. Denoising autoencoders, with Gaussian noise (left) or dropout (right)

Another way to force the autoencoder to learn useful features is to add noise to its inputs, training it to recover the original, noise-free inputs. In a 2010 paper of Vincent et al.¹, the authors introduced the *stacked denoising autoencoders*. The noise can be pure Gaussian noise added to the inputs, or it can be randomly switched-off inputs, just like in *dropout*.

¹Pascal Vincent et al., "Stacked Denoising Autoencoders: Learning Useful Representations in a Deep Network with a Local Denoising Criterion," Journal of Machine Learning Research 11 (2010): 3371-3408.

Denoising Autoencoders (cont.)

An Implementation Example

```
dropout_encoder = tf.keras.Sequential([
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dropout(0.5),
    tf.keras.layers.Dense(100, activation="relu"),
    tf.keras.layers.Dense(30, activation="relu")
])
dropout_decoder = tf.keras.Sequential([
    tf.keras.layers.Dense(100, activation="relu"),
    tf.keras.layers.Dense(28 * 28),
    tf.keras.layers.Reshape([28, 28])
])
dropout_ae = tf.keras.Sequential([dropout_encoder, dropout_decoder])

# compile and fit the model
dropout_ae.compile(loss="mse", optimizer="nadam")
history = dropout_ae.fit(X_train, X_train, epochs=10,
                        validation_data=(X_valid, X_valid))
```



Figure 17-9. (top) Noisy images (with half the pixels turned off) and (bottom) their reconstructions by the dropout-based denoising autoencoder.

Variational Autoencoders

An important category of autoencoders was introduced in 2013 by Diederik Kingma and Max Welling² and quickly became one of the most popular variants: *variational autoencoders* (VAEs).

VAEs are quite different from all the previous autoencoders, in these particular ways:

- They are *probabilistic autoencoders*, meaning that their outputs are partly determined by chance, even after training (as opposed to denoising autoencoders, which use randomness only during training).
- Most importantly, they are *generative autoencoders*, meaning that they can generate new instances that look like they were sampled from the training set.

Training a VAE on a given dataset allows us to get a good approximation of the data distribution. Once we have that, we can sample from it to generate new data instances.

²Diederik Kingma and Max Welling, "Auto-Encoding Variational Bayes", arXiv preprint arXiv:1312.6114 (2013).

Variational Autoencoders (cont.)

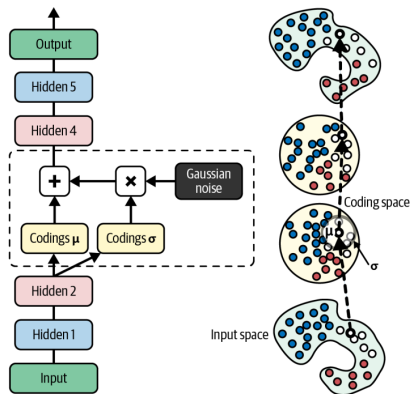


Figure 17-11. A variational autoencoder (left) and an instance going through it (right)

In the diagram, the encoder and decoder have two hidden layers each; however, there is a twist. Instead of learning lower dimensional representations of the inputs to pass on to the decoder, the encoder learns a *mean coding* μ and a *standard deviation coding* σ of these representations. The decoder then draws samples from this distribution (assuming Gaussian) to reconstruct the inputs.

This forces the variational autoencoder to learn the best μ and σ that describes the distribution (assuming Gaussian) of the inputs.

Variational Autoencoders (cont.)

- During training, the cost function (see later) pushes the codings to gradually migrate within the coding space (also called the *latent space*) to end up looking like a cloud of Gaussian points.
- Drawing samples from this latent space thus allows the decoder to reconstruct the inputs (using the two hidden layers *Hidden 3* and *Hidden 4*).
- The *variational autoencoder's latent loss* is defined as:

$$\mathcal{L} = -\frac{1}{2} \sum_{i=1}^n [1 + \log(\sigma_i^2) - \sigma_i^2 - \mu_i^2],$$

where n is the codings' dimension; μ_i and σ_i are, respectively, the i^{th} components of $\boldsymbol{\mu}$ and $\boldsymbol{\sigma}$.

- A common tweak adopted in practice is to make the encoder output γ (defined as $\log(\sigma^2)$) rather than σ , so that the algorithm is more numerically stable and the training is faster. So, the *variational autoencoder's latent loss* becomes

$$\mathcal{L} = -\frac{1}{2} \sum_{i=1}^n [1 + \gamma_i - \exp(\gamma_i) - \mu_i^2].$$

Variational Autoencoders (cont.)

The Loss Function

So, the loss (or cost) function in a VAE consists of two parts:

- The first part is the usual *reconstruction loss* that pushes the autoencoder to reproduce its inputs.
- The second part is the *latent loss* that pushes the autoencoder to have codings that look as though they were sampled from a simple Gaussian distribution: it is the *Kullback-Leibler divergence* between the target distribution (i.e., the Gaussian distribution) and the actual distribution of the codings.

This gives us the formula³ for $\mathcal{L}$ shown on the previous slide.

We will see later on how we add the *latent loss* to the total loss function.

³For more mathematical details, check out the original paper on *variational autoencoders*, or Carl Doersch's "Tutorial on Variational Autoencoders", arXiv:1606.05908, 2016 (2016).

Variational Autoencoders (cont.)

Implementation in Keras

Note: there are quite a few changes to Géron's code below. Check those lines that have "# ***"

```
# define a customized layer
class Sampling(tf.keras.Layer):
    def call(self, inputs):
        mean, logvar = inputs
        # sum the losses over all the features
        loss = -0.5 * tf.reduce_sum(1 + logvar - tf.exp(logvar) - tf.square(mean), axis=-1)
        # average over instances in the minibatch. 784=28*28 is the output image dimension.
        # *** add the latent loss here ***
        self.add_loss(tf.reduce_mean(loss) / 784.) # ***
        return tf.random.normal(tf.shape(logvar)) * tf.exp(logvar / 2) + mean

# the encoder part of the VAE
inputs = tf.keras.Input(shape=[28, 28])
Z = tf.keras.layers.Flatten()(inputs)
Z = tf.keras.layers.Dense(150, activation="relu")(Z)
Z = tf.keras.layers.Dense(100, activation="relu")(Z)
codings_mean = tf.keras.layers.Dense(codings_size)(Z) # mu
codings_log_var = tf.keras.layers.Dense(codings_size)(Z) # gamma
codings = Sampling()(codings_mean, codings_log_var)
variational_encoder = tf.keras.Model(
    inputs=inputs, outputs=[codings_mean, codings_log_var, codings]) # ***

# the decoder part of the VAE
decoder_inputs = tf.keras.Input(shape=[codings_size])
x = tf.keras.layers.Dense(100, activation="relu")(decoder_inputs)
x = tf.keras.layers.Dense(150, activation="relu")(x)
x = tf.keras.layers.Dense(28 * 28)(x)
outputs = tf.keras.layers.Reshape([28, 28])(x)
variational_decoder = tf.keras.Model(inputs=decoder_inputs, outputs=outputs) # ***
```

Variational Autoencoders (cont.)

Implementation in Keras

```
# now join the encoder and decoder together to form our VAE
_, _, codings = variational_encoder(inputs)
reconstructions = variational_decoder(codings)
variational_ae = tf.keras.Model(inputs=inputs, outputs=reconstructions)

# compile and train our VAE to minimize the total loss (latent loss + reconstruction loss)
variational_ae.compile(loss="mse", optimizer="nadam")
history = variational_ae.fit(X_train, X_train, epochs=25, batch_size=128,
                             validation_data=(X_valid, X_valid))
```

Exercise: Run `variational_encoder.summary()` and `variational_decoder.summary()` and inspect the layers in them.

Variational Autoencoders (cont.)

Implementation in Keras

```
# after training, we can generate Fashion MNIST images
codings = tf.random.normal(shape=[3 * 7, codings_size])
images = variational_decoder(codings).numpy() # shape is (21,28,28)
```

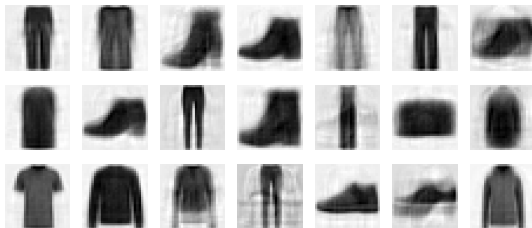


Figure 17-12. Fashion MNIST images generated by the variational autoencoder

The majority of these images look fairly convincing, if a bit too fuzzy. The rest are not great, but the codings are only 10-dimensional and the VAE was trained only for 25 epochs.

Variational Autoencoders (cont.)

Semantic Interpolation Using VAEs

Variational autoencoders make it possible to perform *semantic interpolation*: instead of interpolating two images at the pixel level (which would look as if the two images were overlaid), we can interpolate at the codings level as follows:

- 1 run both images through the encoder to get their codings,
- 2 interpolate the two codings,
- 3 decode the interpolated codings to get the final image.

We can also replace step 1 by directly drawing random samples from the normal distribution to get the two codings.

Variational Autoencoders (cont.)

Semantic Interpolation Using VAEs

For example, let's interpolate 2 codings in the latent space and decode them. We get a sequence of images that gradually go from **T-shirt** to **Sneaker**:

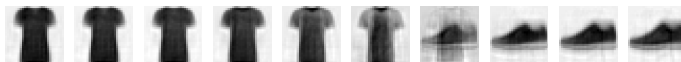
```
tf.random.set_seed(123)

# generate 2 codings from the normal distribution
codings = tf.random.normal(shape=[2, codings_size])

# linearly interpolate 10 elements from the 2 codings
inter_codings = np.linspace(codings[0], codings[1], 10)

# decode the 10 codings
images = variational_decoder(inter_codings).numpy()

plot_multiple_images(images, n_cols=10)           # see code for chapter 17
```



Semantic interpolation.

Generative Adversarial Networks

Generative adversarial networks (GAN) were proposed in 2014 by Ian Goodfellow et al.⁴. Although the idea got researchers excited almost instantly, it took a few years to overcome some of the difficulties of training GANs.

Like many great ideas, it seems simple in hindsight: make neural networks compete against each other in the hope that this competition will push them to excel.

As shown in Figure 17-14, a GAN is composed of two neural networks:

- a *generator*; and
- a *discriminator*.

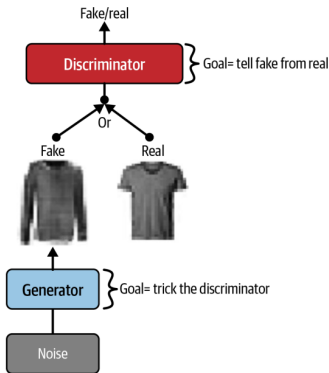


Figure 17-14. A generative adversarial network.

⁴ Ian Goodfellow et al., "Generative Adversarial Nets," Int. Conf. on Neural Information Processing Systems (2014): 2672-2680.

Generative Adversarial Networks (cont.)

A GAN is composed of **two neural networks**:

- **Generator**. It takes a random distribution (typically, a Gaussian) as input and outputs some data (typically, an image). You can think of the random inputs as the *latent representations* (i.e., *codings*) of the image to be generated. So, the generator offers the same functionality as a decoder in a VAE, and it can be used in the same way to generate new images (just feed it some Gaussian noise, and it outputs a new image).
- **Discriminator**. It takes either a fake image from the generator or a real image from the training set as input, and must guess whether the input image is fake or real.

Generative Adversarial Networks (cont.)

During training, the generator and the discriminator have opposite goals: the discriminator tries to distinguish fake images from real images, while the generator tries to produce images that look real enough to trick the discriminator.

Because the GAN is composed of two networks with different objectives, it cannot be trained like a regular neural network. Each training iteration is divided into **two phases**:

- **Phase 1: Training the discriminator.** A batch of real images is sampled from the training set and is completed with an equal number of fake images produced by the generator. The labels are set to 0 for fake images and 1 for real images. The discriminator is trained on this labelled batch for one step, using the *binary cross-entropy loss*. Importantly, backpropagation only optimizes the weights of the discriminator during this phase.

Generative Adversarial Networks (cont.)

- **Phase 2: Training the generator.** We first use the generator to produce another batch of fake images, and once again the discriminator is used to tell whether the images are fake or real. This time we do not add real images in the batch, and all the labels are set to 1 (real); in other words, we want the generator to produce images that the discriminator will (wrongly) believe to be real! Crucially, the weights of the discriminator are frozen during this step, so backpropagation only affects the weights of the generator.

Generative Adversarial Networks (cont.)

We will build a GAN model on the Fashion MNIST dataset. The generator is similar to an autoencoder's decoder, and the discriminator is a regular binary classifier.

Implementing a GAN using Keras

- Construct the network

```
codings_size = 30
from tensorflow.keras.layers import Dense
generator = tf.keras.Sequential([
    Dense(100, activation="relu", kernel_initializer="he_normal"),
    Dense(150, activation="relu", kernel_initializer="he_normal"),
    Dense(28 * 28, activation="sigmoid"),
    tf.keras.layers.Reshape([28, 28])
])
discriminator = tf.keras.Sequential([
    tf.keras.layers.Flatten(),
    Dense(150, activation="relu", kernel_initializer="he_normal"),
    Dense(100, activation="relu", kernel_initializer="he_normal"),
    Dense(1, activation="sigmoid")
])
gan = tf.keras.Sequential([generator, discriminator])
```

Generative Adversarial Networks (cont.)

Implementing a GAN using Keras

- Compile the network

```
discriminator.compile(loss="binary_crossentropy", optimizer="rmsprop")
gan.compile(loss="binary_crossentropy", optimizer="rmsprop")
```

- Create a `tf.data.Dataset` to iterate through the images:

```
batch_size = 32
dataset = tf.data.Dataset.from_tensor_slices(X_train).shuffle(1000)
dataset = dataset.batch(batch_size, drop_remainder=True).prefetch(1)
```

Since the training loop is unusual, we cannot use the regular `fit()` method. Instead, we will write a custom training loop.

Note that there are two training phases in each iteration in the code on the next slide.

Generative Adversarial Networks (cont.)

Implementing a GAN using Keras

- Write our own training loop

```
def train_gan(gan, dataset, batch_size, codings_size, n_epochs):
    generator, discriminator = gan.layers
    for epoch in tf.range(n_epochs):
        epoch = tf.cast(epoch, tf.int64)
        tf.print(f"Epoch {epoch + 1}/{n_epochs}") # extra code
        for X_batch in dataset:
            ### phase 1 - training the discriminator
            noise = tf.random.normal(shape=[batch_size, codings_size])
            generated_images = generator(noise)
            X_fake_and_real = tf.concat([generated_images, X_batch], axis=0)
            y1 = tf.constant([[0.] * batch_size + [[1.]] * batch_size])
            discriminator.trainable = True # *** this line is missing in the author's code
            discriminator.train_on_batch(X_fake_and_real, y1)
            ### phase 2 - training the generator
            noise = tf.random.normal(shape=[batch_size, codings_size])
            y2 = tf.constant([[1.]] * batch_size)
            # discriminator now needs to be non-trainable so that training gan
            # will result in training the generator only.
            discriminator.trainable = False # *** this line is missing in the author's code
            gan.train_on_batch(noise, y2)
        # extra code - plot images during training (omit to save time)
    train_gan(gan, dataset, batch_size, codings_size, n_epochs=50)
```

Note: `train_on_batch` is a method in the `tf.keras.Model` class. It runs a single gradient update on a single batch of data. (For more details, see https://www.tensorflow.org/api_docs/python/tf/keras/Model#train_on_batch)

Generative Adversarial Networks (cont.)

Visualising some generated images

At the end of the first epoch, the generated images already start to look like (very noisy) Fashion MNIST images.



Figure 17-15. Images generated by the GAN after one epoch of training

Unfortunately, the images never really get much better than that, and you may even find epochs where the GAN seems to be forgetting what it learned. Why is that? Well, it turns out that training a GAN can be challenging.

Generative Adversarial Networks (cont.)

The Difficulties of Training GANs

- As training advances, the game may end up in a state that game theorists call a *Nash equilibrium* – when no player would be better off changing their own strategy, assuming the other players do not change theirs. The authors of the paper demonstrated that a GAN can only reach a single Nash equilibrium; however, nothing guarantees that the equilibrium will ever be reached.
- The biggest difficulty is called *mode collapse*: this is when the generator's outputs gradually become less diverse, only producing images in a specific class.
- Moreover, because the generator and the discriminator are constantly pushing against each other, their parameters may end up oscillating and becoming unstable.

In short, this is still a very active field of research, and the dynamics of GANs are still not perfectly understood. The good news is that great progress has been made, and some of the results are truly astounding!

Deep Convolutional GANs

In the architecture of *Deep Convolutional GANs*⁵ (DCGANs), the authors proposed the following guidelines for building stable convolutional GANs:

- Replace any pooling layers with strided convolutions (in the discriminator) and transposed convolutions (in the generator).
- Use Batch Normalization in both the generator and the discriminator, except in the generator's output layer and the discriminator's input layer.
- Remove fully connected hidden layers for deeper architectures.
- Use ReLU activation in the generator for all layers except the output layer, which should use tanh.
- Use leaky ReLU activation in the discriminator for all layers.

These guidelines will work in many cases, but not always.

⁵ Alec Radford et al., "Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks," arXiv:1511.06434 (2015).

Deep Convolutional GANs

Implementing a DCGAN using Keras

```
codings_size = 100

generator = tf.keras.Sequential([
    tf.keras.layers.Dense(7 * 7 * 128),
    tf.keras.layers.Reshape([7, 7, 128]),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.Conv2DTranspose(64, kernel_size=5, strides=2,
                                    padding="same", activation="relu"),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.Conv2DTranspose(1, kernel_size=5, strides=2,
                                    padding="same", activation="tanh"),
])

discriminator = tf.keras.Sequential([
    tf.keras.layers.Conv2D(64, kernel_size=5, strides=2, padding="same",
                           activation=tf.keras.layers.LeakyReLU(0.2)),
    tf.keras.layers.Dropout(0.4),
    tf.keras.layers.Conv2D(128, kernel_size=5, strides=2, padding="same",
                           activation=tf.keras.layers.LeakyReLU(0.2)),
    tf.keras.layers.Dropout(0.4),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(1, activation="sigmoid")
])

gan = tf.keras.Sequential([generator, discriminator])

# compilation and training are similar to the code for GANs
```

Deep Convolutional GANs

Visualising some generated images



Figure 17-16. Images generated by the DCGAN after 50 epochs of training

The images generated after 50 epochs of training still do not look perfect; however, if the architecture is scaled up and trained on a large dataset of faces, one can generate fairly realistic face images.

Summary

- Understand convolutional autoencoders
- Understand denoising autoencoders
- Understand variational autoencoders
- Understand generative adversarial networks
- Understand deep convolutional generative adversarial networks

For the next topic...

Read up to Chapter 16 on **Natural Language Processing with RNNs and Attention** (roughly from pages 595 to 629).

And that's all for this lecture.

Have a good week.